

DECLARATIVE PROGRAMMING

Complete English Edition

José Carpio Cañada · Gonzalo Antonio Aranda Corral · José Marco de la Rosa

Teaching Books [95] · Universidad de Huelva · 2021

INTRODUCTION

Introduction

The material that we present here aims to guide the student in the study of Logical Programming and Functional Programming, both paradigms included within Declarative Programming.

This material has an eminently practical approach. We have reduced the theoretical concepts to a minimum, including only the elements that we consider essential to understand what a declarative program is. We have included the basic theoretical knowledge so that the student can begin to program declaratively from the first session.

The material is divided into fourteen theoretical sessions and nine practical sessions with an approximate duration of an hour and a half per session. The theoretical sessions include exercises that serve to reinforce the theoretical concepts. In the practical sessions, a series of exercises are proposed that students must program using the computer and the Prolog or Haskell compiler or interpreter, as appropriate.

How to use this material?

Understanding the philosophy of declarative programming is not an easy task. It requires a change in the way you think about a program. We went from ordering a series of actions (imperative C, C++, Java or Perl programming) to describing the problem using a series of rules (Prolog and Haskell declarative programming). To understand the declarative programming paradigm we need to reinforce the idea that it is possible to program by changing the focus on building programs. This is achieved progressively, starting with simple exercises, increasing the difficulty until implementing exercises with complex data structures such as trees or graphs.

At the beginning, it can help the student to have the computer close by and try the proposed exercises and see that they really work. It is not always obvious to see that a declarative program works after looking at the code — and what is more important, in what way has the solution been reached. This material uses an approach based on the Mathematical Induction Principle and is supported by this in the process of building the programs. Not all the manuals that we find in the bibliography highlight this idea. Rather, they use an approach similar to the description of other imperative

languages, language syntax, and sets of functions, methods, or predicates. We think that in declarative programming it is essential to understand that it is necessary to change the approach when programming and we place special emphasis on this. The idea that from the smallest case ($n-1$) we deduce the generic case (n) is repeated in most sessions.

After studying this material, if the student has understood how to build the programs, he will not need to mentally follow the execution of the program to know if it works. Based on the principle of induction, the student should be able to tell whether the program can work or not after looking at the code. From this moment on, the exercises that at first seemed impossible to understand or program become understandable from the declarative perspective.

We hope that this is the case and that after studying this material the students understand that programs can be made in a different way.

Acknowledgements

We want to thank the students who attended our classes with attention and interest by contributing many ideas that are now reflected in this teaching manual. To our colleague Manuel Maestre Hachero who kindly offered to review the material and correct it from the vision of a mathematician. To Francisco Moreno Velo, Marcos del Toro Peral, Águeda López Moreno and Diego Muñoz Escalante who taught this course with us and contributed their ideas. To professors Jim Royer and Susan Older from the University of Syracuse in the United States who kindly allowed us to include the exercises from Haskell's second session of this manual.

Authors

José Carpio Cañada

Gonzalo Antonio Aranda Corral

José Marco de la Rosa

Contents

Theory

Functional Programming with Haskell

0. Introduction
1. Definition of functions
2. Operator precedence
3. Lazy evaluation
4. Higher-order functions

5. Currying
6. Function composition
7. Lists
8. Patterns
9. Tuples
10. Recursion

Logic Programming with Prolog

0. Introduction
1. Unification
2. Data types
 - 2.1 Lists
 - 2.2 Trees
 - 2.3 Graphs
3. Execution control
4. State problems

Practical Sessions

Functional Programming with Haskell

- Introduction to the Haskell Hugs programming environment
- Lists in Haskell
- Patterns, tuples, recursion and extended list notation in Haskell
- Haskell exercises from previous exams

Logic Programming with Prolog

- Introduction to the SWI-Prolog environment
- Simple predicates in Prolog
- Predicates on lists in Prolog
- Trees in Prolog
- Graphs in Prolog

THEORY

INTRODUCTION TO DECLARATIVE PROGRAMMING

Section 0. Introduction

The objective of this course is to introduce the student to a form of programming based on a paradigm very different from the more usual imperative paradigm. This

task will help the student improve their capacity for abstraction and to practise with programming concepts such as recursion or lazy evaluation, rare in other programming subjects. The declarative paradigm also provides the student with a conceptual environment frequently used for the study of certain artificial intelligence problems.

Section 1. Programming Paradigms

Paradigm (according to the Merriam-Webster dictionary): "A philosophical and theoretical framework of a scientific school or discipline within which theories, laws and generalizations, and the experiments performed in support of them are formulated; broadly: a philosophical or theoretical framework of any kind."

Computer programming can be defined as the creation of coded descriptions that a computer can interpret to solve a certain problem. The programming task will depend entirely on the code interpretation capacity of the computer in question since we will have to adapt the generated descriptions to the language understood by the machine. Over time we have arrived at a very diverse range of languages created incrementally on top of the original machine language. Each programming language addresses problem solving based on a set of concepts and a vision of how the computer should behave. Each such set of concepts is called a programming paradigm.

Examples of different programming paradigms:

- Imperative programming
- Structured programming
- Declarative programming
- Logic programming
- Functional programming
- Event-driven programming
- Modular programming
- Aspect-oriented programming
- Object-oriented programming
- Constraint-based programming

Section 2. Imperative Programming

The imperative paradigm understands that to solve a problem a series of steps must be carried out and the programmer is in charge of describing in an orderly and systematic way the steps that the computer must follow. Examples of imperative languages: BASIC, C, Fortran, Pascal, Perl, PHP.

Program example — sorting with the bubble algorithm:

```
void exchange(int *x, int *y) { int temp; temp=*x; *x=*y; *y=temp; }
void bubble(int list[], int n) {
    int i, j;
    for(i=0; i<(n-1); i++)
        for(j=0; j<(n-(i+1)); j++)
            if(list[j] > list[j+1]) exchange(&list[j], &list[j+1]);
}
void main() { /* ... */ bubble(list, LIST_ELEMENTS); /* ... */ }
```

Section 3. Declarative Programming

The declarative paradigm proposes that problems be described to the computer with a series of basic conceptual units that can be combined according to certain rules to generate new information. To solve a problem, descriptions that represent the problem domain are created and the problem is posed as a question. The conceptual units depend on the specific sub-paradigm. In this course we study two sub-paradigms: functional programming and logic programming.

Advantages:

- Compact and very expressive descriptions. It is possible to describe universes of problems with very few lines of code.
- Development not so oriented to the solution of a single problem. With proper programming, it is enough to describe a problem domain correctly and know how to formulate our problem as a simple query in that domain.
- There is no need to design an algorithm that solves the problem.

3.1 Functional Programming

Functional programming uses the function as a basic descriptive concept. In this subject we will study the Haskell language.

<http://www.haskell.org/haskellwiki/Introduction>

Program example — checking that a number is natural:

```
natural :: (Num a, Ord a) => a -> Bool
natural 1 = True
natural n | n > 1 = natural (n-1) | otherwise = False
Main> natural (-1)    False :: Bool
Main> natural (10)   True  :: Bool
```

3.2 Logic Programming

Logic programming uses the logical predicate as its basic descriptive concept. In this subject we will study the Prolog language, specifically the free software version SWI-Prolog. <http://www.swi-prolog.org/>

Program example — descent relationships:

```
% father(Father, Son)
father(joan, peter).  father(peter, luis).  father(iker, yeray).
% descendant(Descendant, Ancestor)
descendant(D, A) :- father(A, D).
descendant(D, A) :- father(A, Aux), descendant(D, Aux).
?- father(joan, X).  X = peter.
?- descendant(luis, peter).  true.
?- descendant(X, joan).  X=peter ; X=luis ; false.
```

Section 4. Bibliography

General bibliography

Programming in Prolog — W.F. Clocksin, C.S. Mellish. Springer Verlag, 1994.

Prolog Programming for Artificial Intelligence — Ivan Bratko. Addison Wesley, 1990.

Reasoning with Haskell — Blas C. Ruíz, F. Gutiérrez, et al. Thompson, 2004.

Specific bibliography

Prolog: The Standard — P. Deransart, A. Ed-Dbali, L. Cervoni. Springer, 1996.

An Introduction to Computing in Haskell — M.M.T. Chakravarty, G.C. Keller.
Pearson SprintPrint, 2002.

Programming Languages: Principles and Paradigms — A. Tucker, R. Noonan.
McGraw-Hill, 2003.

FUNCTIONAL PROGRAMMING WITH HASKELL

Section 0. Introduction

What is functional programming?

Languages such as C, Pascal, and Java are imperative languages — when we program we specify HOW to solve a problem. The goal in functional programming is to define WHAT TO COMPUTE, but not how to compute it. An example is a spreadsheet: we do not specify the order in which cells are computed, nor how to organise memory. We specify the value of a cell using an expression but not the sequence of steps needed.

In a spreadsheet we do not know when assignments are performed, so we cannot make use of them. This is a fundamental difference from imperative languages such as C or Java, where careful specification of assignments is required.

Advantages of functional programming

- More concise programs
- Easier to understand
- No core dumps

Characteristics of Haskell

Haskell is a purely functional programming language with static polymorphic types and lazy evaluation. Its name comes from the mathematician Haskell Brooks Curry, who specialised in mathematical logic. Haskell is based on the lambda calculus.

- Type inference — type declarations are optional
- Lazy evaluation — data are only computed when required
- Compiled and interpreted versions
- Everything is an expression
- Functions can be defined anywhere, used as arguments, and returned as results

Haskell implementations

Various implementations exist: GHC, Hugs, nhc98 and Yhc. For the practical sessions we will use Hugs (<http://haskell.org/hugs/>).

Implementation	Error messages	Size	Tools	Notes
Hugs	+/-	++	-	Widely used for learning. Fast compilation. Rapid code development.
GHC	+	-	++	Generated code is very fast. Possibly the most widely used implementation.
NHC	?	+	++	Supports profiling, debugging, tracing.
Yhc	?	+	?	Still in development.
Helium	++	++	-	Created for teaching.

What will we NOT cover about Haskell?

- We will not look at examples of connecting Haskell to other languages.

When is programming in C better?

In general C is faster and may use much less memory. Haskell needs more memory and is slower. In applications where speed is important C may be a better choice.

Can Haskell connect to other programming languages?

HaskellDirect is a tool based on IDL that allows Haskell programs to work with software components. It is possible to use C/C++ functions from Haskell using Green Card or C->Haskell.

Section 1. Function Definitions / Local Definitions

```
{-}
-- DECLARATION
nonNegative :: (Num a, Ord a) => a -> Bool
{- PURPOSE: Returns True if x >= 0, False otherwise -}
-- DEFINITION
nonNegative x = x >= 0
{- TESTS: test1 = nonNegative (-2.5) -- False
          test2 = nonNegative 0      -- True
          test3 = nonNegative 5      -- True -}
```

Elements needed to define a function:

- A comment block: {- ... -} for blocks, -- for single lines.
- A function header: <function_name> :: <type_declaration>. The function name begins with a lower-case letter.
- To define types, use type variables from a class (Eq, Ord, Enum, Num, Fractional, etc.) or basic types (Int, Integer, Float, Double, Char, etc.).
- Haskell can infer types automatically. For exercises in this course, function headers are mandatory.

Examples of data types used by operators

a) "==" requires the Eq class:

```
(==) :: Eq a => a -> a -> Bool
```

b) ">" requires the Ord class:

```
(<) :: Ord a => a -> a -> Bool
```

c) "+" requires the Num class:

```
(+) :: Num a => a -> a -> a
```

Type classification examples

a) Identity function:

```
identity :: a -> a
identity x = x
```

b) All-equal function:

```
equal :: Eq a => a -> a -> a -> Bool
equal x y z = x==y && y==z
```

c) Divide function:

```
divide :: Fractional a => a -> a -> a
divide x y = x / y
```

Ways to define functions

- Using multiple equations
- Guards ("|")
- If-then-else
- Case
- Local definitions

1.1) Multiple equations

```
factorial :: Int -> Int
factorial 0 = 1
factorial n = n * factorial (n-1)
```

1.2) Guards

```
factorial n
  | n == 0    = 1
  | n > 0     = n * factorial (n-1)
  | otherwise = error "negative value"
```

1.3) If-then-else

```
factorial n = if (n==0) then 1 else n * factorial (n-1)
```

1.4) Case

```
translate x = case x of
  1 -> "A"
  2 -> "B"
  3 -> "C"
```

1.5) Local definitions

```
divisible :: Int -> Int -> Bool
divisible x y = remainder == 0
  where remainder = mod x y
```

Important: the local definition must be indented to the right of where the function definition starts.

Section 2. Operator Precedence

The following table defines the precedences of the operators in the Prelude module:

Notation	Priority	Operator
infixr	9	.
infixl	9	!!
infixr	8	^, ^^, **
infixl	7	*, /, `quot`, `rem`, `div`, `mod`
infixl	6	+, -
infixr	5	:
infixr	5	++
infix	4	==, /=, <, <=, >=, >, `elem`, `notElem`
infixr	3	&&
infixr	2	
infixl	1	>>, >>=
infixr	1	=<<
infixr	0	`, \$!, `seq`

The infix notation means the operator is written between the operands. When chaining operators of the same precedence, no tie-breaking order is defined — parentheses must be used explicitly.

infixl: when precedences are equal, the left operand is evaluated first (e.g. $1-2-1 = -2$).

infixr: when precedences are equal, the rightmost operator is evaluated first (e.g. $2^1^2 = 2$).

Functions always have higher priority than operators. The composition operator "." has the highest precedence of all operators.

```
Hugs> (succ . pred) 4    →    4 :: Integer
max x y = fromIntegral (x + y + abs (x - y)) / 2.0
```

Section 3. Lazy Evaluation

Languages that use this technique only evaluate an expression when it is needed:

```
onlyFirst :: a -> b -> a
onlyFirst x _ = x
Main> onlyFirst 4 (7/0)    →    4 :: Integer
```

The expression `7/0` in Haskell has the value `Infinity`. Evaluating the expression above would cause an error in most imperative languages; however in Haskell no error occurs because the second argument is never evaluated — it is not needed.

Section 4. Higher-Order Functions

Higher-order functions take one or more functions as input arguments and/or return a function as output. An example is `map` from the `Prelude` module:

```
map :: (a -> b) -> [a] -> [b]
map _ [] = []
map f (head:tail) = f head : map f tail
Main> map succ [1,2,3] → [2,3,4] :: [Integer]
```

Section 5. Currying

Currying consists of calling a function using only some of its leftmost parameters. Named after Haskell Brooks Curry.

```
add :: Int -> Int -> Int -> Int
add x y z = x + y + z
```

- `add 1` → returns a function that receives two integers and returns one
- `add 1 2` → returns a function that receives one integer and returns one
- `add 1 2 3` → returns the integer 6

```
Main> map (add 1 1) [1,2,3] → [3,4,5] :: [Int]
```

Section 6. Function Composition

```
f . g $ x = f(g(x))
```

Haskell provides the dot operator `(.)`: `(.) :: (a->b) -> (c->a) -> c->b`. The most important constraint is that the output type of `g` must match the input type of `f`.

```
Main> ((== True) . (< 0)) 5 → False :: Bool
```

Section 7. Lists

Haskell provides a convenient mechanism for defining lists: `[1..10]`, `[15..20]`, `['q'..'z']`, `[14..2]`, etc.

Extended list notation

Consists of three parts: 1) Generator, 2) Restrictions (optional), 3) Transformation.

```
[ (x, True) | x <- [1..20], even x, x < 15 ]
```

Generator: `x <- [1..20]`; Restrictions: `even x`, `x < 15`; Transformation: `(x, True)`. This generates a list of pairs `(x, True)` where `x` is between 1 and 20, is even, and less than 15.

Most common list functions

Function header	Description
<code>(:) :: a -> [a] -> [a]</code>	Adds an element at the beginning of a list
<code>(++) :: [a] -> [a] -> [a]</code>	Concatenates two lists
<code>(!!) :: [a] -> Int -> a</code>	Returns the nth element
<code>null :: [a] -> Bool</code>	Returns True if the list is empty
<code>length :: [a] -> Int</code>	Returns the number of elements
<code>head :: [a] -> a</code>	Returns the first element
<code>tail :: [a] -> [a]</code>	All elements except the first
<code>take :: Int -> [a] -> [a]</code>	Takes the first n elements
<code>drop :: Int -> [a] -> [a]</code>	Removes the first n elements
<code>reverse :: [a] -> [a]</code>	Reverses the order of elements
<code>zip :: [a] -> [b] -> [(a,b)]</code>	Creates a list of pairs
<code>unzip :: [(a,b)] -> ([a],[b])</code>	From a list of pairs generates two lists
<code>sum :: Num a => [a] -> a</code>	Sums the elements of a list
<code>product :: Num a => [a] -> a</code>	Multiplies the elements of a list

Reference: <http://www.cs.ut.ee/~varmo/MFP2004/PreludeTour.pdf>

Section 8. Patterns

Possible implementation of map:

```
map :: (a -> b) -> [a] -> [b]
map _ [] = []
map f (head:tail) = f head : map f tail
```

Pattern types: `[]` matches an empty list; `(head:tail)` matches a non-empty list; `f` matches any value and binds it; `_` matches any value and ignores it. Pattern matching is both a way of binding values to variables and a way of decomposing expressions.

What can we use as a pattern?

Only constructors and constants (1, 2, True, False, etc.) are allowed in patterns. Operators like "++" are not allowed.

```
removeThree :: [a] -> [a]
removeThree (_:_:_:rest) = rest
```

Exception: the (n+k) pattern

```
predecessor :: Int -> Int
predecessor (n+1) = n
```

Only integers are allowed, and the value must be $\geq k$.

Pattern aliases

Use `name@<pattern>` to access the full expression while also decomposing it:

```
trimLeft :: String -> String
trimLeft string@(head:rest)
  | head == ' ' = trimLeft rest
  | otherwise   = string
```

Section 9. Tuples

We can group expressions of different types into a tuple: `(1,2)`, `('a',1,"Hello")`, `((1,2),[3,4],(0,'a'))`, `((+),7,(*))`.

Functions with tuples:

```
first  (x, y)    = x
first2 (x, y, z) = x
```

Defining types with tuples

```
type Entry      = (Person, Age, Phone)
type Directory = [Entry]
find :: Directory -> Person -> [Phone]
find list person = [phone | (per, age, phone) <- list, person == per]
```

Section 10. Recursion

In Haskell there are no loops. The way to "iterate" is by using recursion. Recursion relies on the principle of mathematical induction:

- (1) The statement `P` is true for an initial value `n0` ("base case").
- (2) `P` is true for a value `n > n0` if it is true for `n-1`.

```
natural :: (Num a, Ord a) => a -> Bool
natural 1 = True
natural n | n > 1 = natural (n-1) | otherwise = False
```

Recursion with lists

The base case for lists is usually the empty list. The `head:tail` pattern separates the first element from the rest. Implementing `numElements`:

```
numElements [] = 0
numElements (head:tail) = 1 + numElements tail
```

Implementing `foldl`

`foldl :: (a -> b -> a) -> a -> [b] -> a`. Joins the elements of a list using a binary operator and an initial element, using left association.

```
foldl f el [] = el
foldl f el (head:tail) = foldl f (f el head) tail
Main> foldl (++) "com" ["po","se"] → "compose" :: [Char]
```

How to check if a recursive function is correct

1. Is the base case true?

2. Does the recursive case tend towards the base case?
3. Are all cases covered exactly once?
4. From the recursive output (n-1), can I build the general output (n)?

LOGIC PROGRAMMING WITH PROLOG

Section 0. Introduction

Propositional Logic

We write rules to indicate that if an antecedent occurs then a consequent will follow. A clause is a rule or a fact. With clauses we create a knowledge base. A logic program = Knowledge base + Queries.

Prolog syntax: rules are written starting with the consequent. All clauses end with a period ".".

```
plants_are_watered :- it_rains.
it_rains.
```

Horn clauses: a Horn clause has a conjunction of zero or more terms in the antecedent and a single term in the consequent. The resolution algorithm is decidable if all clauses are Horn clauses.

```
are_watered(tomatoes) :- it_rains.
are_watered(lettuce)  :- it_rains.
are_watered(strawberries) :- it_rains.
it_rains.
?- are_watered(Vegetable).
Vegetable = tomatoes ; Vegetable = lettuce ; Vegetable = strawberries
```

Closed-world hypothesis: everything that is not defined is false.

Variables

Variables are written starting with a capital letter. A variable in Prolog can be instantiated with any data type. Once instantiated, a variable does not change its value. The scope of a variable is restricted to a single clause.

Examples: Person, List, X, Tree, _

```
?- X=1, X=2. No
?- Person = peter. Persona = peter
```

Anonymous variable "_": matches any value but does not bind it. Use it to suppress Singleton variable warnings.

Section 1. Unification

Two terms unify: 1) if they have no variables, they unify if they are identical; 2) if they have variables, they unify if a substitution can be found that makes them identical.

```
?- X = 1.           X=1      Yes
```

```

?- X = woman(maria). X=woman(maria) Yes
?- X = 2+3.          X=2+3    Yes
?- X=1, X=2, X=Y.    No
?- (1+2) = 1+2.      Yes
?- 1+1 = +(1,1).     Yes
?- (1+1)+1 = 1+(1+1). No

```

Operators

"=" (unification): does not evaluate arithmetic. "==" (identity): true if exactly the same.
"is" (arithmetic evaluator): evaluates right-hand side and unifies left. "=:=" (arithmetic comparator): evaluates both sides and compares.

```

?- X = 1+1. X=1+1 (not evaluated)
?- X is 1+1. X=2
?- 2 =:= 1+1. Yes

```

Operators table

Precedence	Notation	Operator
1200	xfx	:-
1000	xfx	,
900	fy	\+
700	xfx	<, =, =.., =:=, =<, ==, =\=, >, >=, is
500	yfx	+, -, xor
500	fx	+, -, \
400	yfx	*, /, mod, rem
200	xfx	*
200	xfy	^

Section 2. Data Types

2.1 Lists

Lists are denoted with brackets, elements separated by commas. They contain terms (variables, constants, or structures). The empty list is []. Lists can contain other lists. The "|" operator separates the first element from the rest:

```

?- [Head|Tail] = [1,2,3]. Head=1, Tail=[2,3]
?- [Head|Tail] = [1].      Head=1, Tail=[]
?- [H1,H2|Tail] = [1,2,3]. H1=1, H2=2, Tail=[3]
?- [_|_] = []. No

```

Implementing num_elem/2 that counts elements in a list:

```

% num_elem(+List, -Int)
num_elem([], 0).
num_elem([_|Tail], NumList) :-
    num_elem(Tail, NumTail),
    NumList is NumTail + 1.

```

Predefined list predicates: append/3, member/2, reverse/2, nth0/3, nth1/3, length/2.

Reversible predicates: Prolog predicates can be used in both directions — providing inputs and expecting outputs, or providing outputs and having Prolog find valid inputs.

List algorithms (implementation examples):

1) Reverse a list

```
% my_reverse(+List, -ListR)
my_reverse([], []).
my_reverse([Head|Tail], RT) :- my_reverse(Tail, R), append(R, [Head], RT).
```

2) Append two lists

```
% my_append(+List1, +List2, -ListR)
my_append([], L, L).
my_append([Head|Tail], List2, [Head|R]) :- my_append(Tail, List2, R).
```

3) Bubble sort

```
% bubble_sort(+List, -ListR)
bubble_sort(List, List) :- my_sort(List).
bubble_sort(List, RT) :- append(Ini, [E1,E2|End], List), E1>E2,
    append(Ini, [E2,E1|End], R), bubble_sort(R, RT).
my_sort([]). my_sort([_]).
my_sort([H1,H2|T]) :- H1 <= H2, my_sort([H2|T]).
```

4) Insertion sort

```
% insert_in_sorted_list(+Elem, +List, -ListR)
insert_in_sorted_list(Elem, [], [Elem]).
insert_in_sorted_list(Elem, [Head|Rest], [Elem,Head|Rest]) :- Elem <= Head.
insert_in_sorted_list(Elem, [Head|Rest], [Head|R]) :-
    Elem > Head, insert_in_sorted_list(Elem, Rest, R).
insertion_sort([], []).
insertion_sort([Head|Rest], RT) :-
    insertion_sort(Rest, R), insert_in_sorted_list(Head, R, RT).
```

5) Quicksort

```
% divide(+Elem, +List, -Smaller, -Larger)
divide(_, [], [], []).
divide(Elem, [Head|Rest], Smaller, [Head|Larger]) :-
    Head > Elem, divide(Elem, Rest, Smaller, Larger).
divide(Elem, [Head|Rest], [Head|Smaller], Larger) :-
    Head <= Elem, divide(Elem, Rest, Smaller, Larger).
% quick_sort(+List, -ListR)
quick_sort([], []).
quick_sort([Head|Rest], R) :-
    divide(Head, Rest, Smaller, Larger),
    quick_sort(Smaller, RSm), quick_sort(Larger, RLg),
    append(RSm, [Head|RLg], R).
```

6) Finding primes between X and Y

```
% list_divisors(+X, +Y, -ListR)
list_divisors(_, 1, [1]).
list_divisors(X, Y, [Y|R]) :- Y>1, Y2 is Y-1, list_divisors(X,Y2,R),
    0 is X mod Y.
list_divisors(X, Y, R) :- Y>1, Y2 is Y-1, list_divisors(X,Y2,R),
    Z is X mod Y, Z\==0.
% prime(+X)
```

```

prime(X) :- list_divisors(X, X, [X,1]).
% primes_between(+X, +Y, -ListR)
primes_between(X, X, []).
primes_between(X, Y, [X|R]) :- X<Y, X2 is X+1,
primes_between(X2,Y,R), prime(X).
primes_between(X, Y, R) :- X<Y, X2 is X+1,
primes_between(X2,Y,R), \+ prime(X).

```

7) Permuting the elements of a list

```

% select_one(+List, -Elem, -Rest)
select_one([Head|Rest], Head, Rest).
select_one([Head|Tail], E, [Head|R]) :- select_one(Tail, E, R).
% permute(+List, -ListR)
permute([], []).
permute(L, [E|RP]) :- select_one(L, E, R), permute(R, RP).

```

8) Finding the most frequent element

```

% most_frequent(+List, -Elem, -Num)
most_frequent([], _, 0).
most_frequent([Head|Tail], Head, N2) :- most_frequent(Tail,Elem,N),
Head=Elem, N2 is N+1.
most_frequent([Head|Tail], Elem, N) :- most_frequent(Tail,Elem,N),
Head\=Elem.

```

Section 2. Data Types (cont.)

2.2 Trees

Binary Trees

A binary tree is either a null tree or a structure with an element and two binary-tree successors. We represent a null tree with 'nil' and a non-empty tree with a(Label, LeftSubtree, RightSubtree).

```

A1 = a(a, a(b,a(d,nil,nil),a(e,nil,nil)),
a(c,nil,a(f,a(g,nil,nil),nil)))
/* count_nodes(+Binary_tree, ?Num_nodes) */
count_nodes(nil, 0).
count_nodes(a(_,LS,RS), R) :- count_nodes(LS,RL), count_nodes(RS,RR),
R is RL+RR+1.
/* list_leaves(+Binary_tree, ?Leaf_list) */
list_leaves(nil, []).
list_leaves(a(_,nil,RS), LD) :- RS\=nil, list_leaves(RS,LD).
list_leaves(a(_,LS,nil), LL) :- LS\=nil, list_leaves(LS,LL).
list_leaves(a(_,LS,RS), LR) :- LS\=nil, RS\=nil,
list_leaves(LS,LL), list_leaves(RS,LD), append(LL,LD,LR).
list_leaves(a(Label,nil,nil), [Label]).

```

Generic Trees

A generic tree is composed of a root and a list of successor trees. Represented as a(Label, ChildList). Implementation follows two groups of clauses: one for a single tree, one for a list of trees.

```

/* count_nodes(+Generic_tree, ?Num_nodes) */
count_nodes(a(_,Children), R) :- count_nodes(Children, N), R is N+1.
count_nodes([], 0).

```



```

count_nodes([Head|Rest], R) :- count_nodes(Head,NH),
count_nodes(Rest,NR), R is NH+NR.
/* depth_gt(+Generic_tree, ?Depth) */
depth_gt(a(_,Children), R) :- depth_gt(Children,DC), R is DC+1.
depth_gt([], 0).
depth_gt([Head|Rest], DH) :- depth_gt(Head,DH), depth_gt(Rest,DR), DH
>= DR.
depth_gt([Head|Rest], DR) :- depth_gt(Head,DH), depth_gt(Rest,DR), DH
< DR.

```

Section 2. Data Types (cont.)

2.3 Graphs

A graph is a set of nodes and edges. Reference: Ninety-Nine Prolog Problems:
<https://prof.ti.bfh.ch/hew1/informatik3/prolog/p-99/>

Representations

Edge-Clause representation (one fact per edge): edge(h,g). edge(k,f). ... — cannot represent isolated nodes.

Graph-Term representation (default): graph([b,c,d,f,g,h,k],
[a(b,c),a(b,f),a(c,f),a(f,k),a(g,h)])

Labelled graphs add information to nodes or edges:

graph([k,m,p,q],[edge(m,p,7),edge(p,m,5),edge(p,q,9)])

Building a path

The path predicate: path(+Start, +End, +Graph, +Visited, ?Path, <?Weight>,
<?Path2>)

Base case: path(End, End, _, []).

Recursive case:

```

path(Start, End, graph(_,Edges), Visited, [edge(Start,TMP)|Path]) :-
    connected(Start, TMP, Edges),
    \+ visited(Start, TMP, Visited),
    path(TMP, End, [edge(Start,TMP)|Visited], Path).

```

Connected and visited predicates

```

% Undirected graph:
connected(S,E,A) :- member(edge(S,E),A). connected(S,E,A) :-
member(edge(E,S),A).
visited(S,E,V) :- member(edge(S,E),V). visited(S,E,V) :-
member(edge(E,S),V).
% Directed graph:
connected(S,E,A) :- member(edge(S,E),A).
visited(S,E,V) :- member(edge(S,E),V).

```

Road map example

```

datum(graph([madrid, cordoba, sevilla, jaen, granada, huelva, cadiz],
[edge(huelva, sevilla, a49, 94), edge(sevilla, cadiz, ap4, 125),
edge(sevilla, granada, a92, 256), edge(granada, jaen, a44, 97),
edge(sevilla, cordoba, a4, 138), edge(jaen, madrid, a4, 335),
edge(cordoba, madrid, a4, 400)])).

```

```
% datum(G), path(huelva, cordoba, G, [], Ca, C, K).
% Ca=[huelva,sevilla,cordoba], C=[a49,a4], K=232 ;
% Ca=[huelva,sevilla,granada,jaen,madrid,cordoba], K=1182.
```

Section 3. Execution Control

The Cut

The cut is denoted by "!". Given: $H :- B_1, \dots, B_m, !, B_{m+1}, \dots, B_n$ (invoked by goal G):

- First effect: solutions found for $B_1, \dots, B_m$ are kept; all alternatives for those goals are discarded.
- Second effect: any attempt to satisfy G with another clause is discarded.

Green cuts vs Red cuts

Green cuts do not modify the set of solutions (they only improve efficiency). Red cuts change the solutions (removing the cut changes the results) — they make programs less declarative and should be used with caution.

Example (red cut):

```
count_leaves(nil, 0).
count_leaves(a(_,nil,nil), 1) :- !.
count_leaves(a(_,LS,RS), R) :- count_leaves(LS,RL),
count_leaves(RS,RR), R is RL+RR.
```

Resolution Trees

Also known as deduction trees, they represent how Prolog solves a query. Algorithm: repeatedly search the knowledge base top-down for unifications; open branches for each match; resolve left to right; mark branches as success or failure; apply cut effects when encountered.

The 'repeat' term

The term repeat has infinitely many solutions, making it useful for infinite loops:

```
p(X) :- write('hello'), repeat, write('goodbye'), read(X), X='end',
!.
```

The 'fail' element

When fail is reached in a clause, that clause becomes false.

SLD-Draw

The tool SLD-Draw (<http://www.lcc.uma.es/~pacog/sldDraw/>) can be used to draw resolution trees without 'repeat'.

Section 4. State Problems

A method for solving problems using logic programming consists of tracing a path through different states until a solution is reached. Transitions between states are made by performing moves.

Solution built in four steps: 1) Define states; 2) Implement moves; 3) Implement path; 4) Implement solution predicate.

Example: Missionaries and Cannibals

Problem: 3 missionaries and 3 cannibals must cross a river using a boat that carries at most 2. Cannibals must never outnumber missionaries on either bank.

```
/* state(Cannibals_left, Missionaries_left, Boat_pos) */
initial(state(3,3,left)). final(state(0,0,right)).
/* cross(?Num_missionaries, ?Num_cannibals, ?Side) */
cross(1,0,left). cross(1,0,right). cross(0,1,left).
cross(0,1,right).
cross(2,0,left). cross(2,0,right). cross(0,2,left).
cross(0,2,right).
cross(1,1,left). cross(1,1,right).
/* mov(?Move, ?Prev_state, ?Next_state) */
mov(cross(M,C,left), state(MI,CI,right), state(MD,CD,left)) :-
    cross(M,C,left), NT is M+C, NT=<2, NT>=1,
    M=<MI, C=<CI, MD is MI-M, CD is CI-C.
mov(cross(M,C,right), state(MI,CI,left), state(MD,CD,right)) :-
    cross(M,C,right), NT is M+C, NT=<2, NT>=1,
    M=<MI, C=<CI, MD is MI-M, CD is CI-C.
/* path(+Initial, +Final, +Visited, -Path) */
path(Start, Start, _, []).
path(Start, End, Visited, [Move|Path]) :-
    length(Visited,L), L<10, mov(Move,Start,Int),
    \+ member(Int,Visited), path(Int,End,[Int|Visited],Path).
solution(Path) :- initial(Ei), final(Ef), path(Ei, Ef, [Ei], Path).
```

Reference: Problem Solving in Prolog — Bratko, Chapter 12.

PRACTICAL SESSIONS — HASKELL

Practice 1: Introduction to the Haskell Hugs Environment

In this first session we will familiarise ourselves with the Hugs programming environment and look at the basic data types.

Section 1. User Account

Each student may have a user account with associated disk space. Use this space only for temporary storage. After each session, save your files to a USB stick or e-mail them to yourself.

Section 0. Getting Started

- Select the Windows XP operating system at startup.

- Enter the login provided by the teacher. The first time you start the system you must change the password.

Section 1. Running Hugs

- From the Windows "Start" menu, select WinHugs.
- From the Hugs window, load programs and run queries. Type an expression and press Enter to see its result.

```
Hugs> 1+1    →    2
Hugs> :type $$    →    "1+1 :: Num a => a"
```

Section 2. Writing a Program

To write a program use a plain-text editor (Windows Notepad works). Create the directory H:\declarative. The simplest way to write a new program:

```
Hugs> :edit "H:\\declarative\\sum.hs"
```

Save and close the window. Then load:

```
Hugs> :load "H:\\declarative\\sum.hs"
Hugs> :info sum    →    sum :: Int -> Int -> Int
```

Section 4. Data Types

The Num class includes Int, Integer, Float and Double. The Fractional class includes Float and Double. Example error when type header does not match inferred type:

```
ERROR: Type error - divide :: Int->Int->Int does not match Int->Int->Float
```

Solution a: keep header and use fromIntegral. Solution b: change header to Fractional. The Hugs> prompt means no module is loaded; Main> means a file is loaded.

Section 3. Summary of Hugs Commands

| Command | Description |
|----------------|--------------------------------------|
| :t / :type | Shows the data type of an expression |
| :? | Shows the list of basic commands |
| \$\$ | Last expression entered |
| :load | Load a file |
| :edit | Opens the default editor |
| :r / :reload | Reloads loaded modules |
| :set | Shows list of options |
| :set +/-option | Enable or disable an option |
| :set +t | Show data types |
| :q | Exit Hugs |

Exercises

5. Implement double: receives a number and returns its double.
6. Implement square: receives a number and returns its square.
7. Implement isPositive: true when its argument is greater than 0.
8. Given $\max(x,y)=((x+y)+|x-y|)/2$, implement a function returning the larger of two Float values.
9. Using the previous function, compute the maximum of three numbers.
10. Implement between0and9: returns True if the integer is between 0 and 9.
11. Implement isMultipleOf3: returns True if the integer is a multiple of 3.
12. Define allFourEqual :: Int->Int->Int->Int->Bool: True if all four arguments are equal.
13. Use the previous function to create allThreeEqual for three integers.
14. Define howManyEqual :: Int->Int->Int->Int: returns how many of three inputs are equal.
15. Implement the function from exercise 4 using only integer values.

Practice 2: Lists in Haskell

Section 1. Lists

In Haskell a list is a sequence of values of the same type. Haskell allows infinite lists such as [1..]. A String is a list of Char. The expression [m..n] generates [m, m+1, ..., n]. If $n < m$, returns an empty list. [m,p..n] generates [m, m+(p-m), m+2(p-m), ...].

a) [1..10] b) [15..20] c) [15..(20-5)] d) ['q'..'z'] e) [14..2]

Section 2. Extended List Notation

Three parts: Generator, Restrictions (optional), Transformation.

```
[(x,True) | x <- [1..20], even x, x < 15]
```

Variables are defined in the generator. Using a variable not defined there causes an error.

```
addPairs :: [(Int,Int)] -> [[Int]]
addPairs pairList = [[m+n] | (m,n) <- pairList, n == 2*m]
allEven list = (list == [x | x <- list, x `mod` 2 == 0])
allOdd list = ([ ] == [x | x <- list, x `mod` 2 == 0])
```

Section 3. Using Templates with Lists

```
doubleAll :: [Int] -> [Int]
doubleAll list = [2*x | x <- list]
doubleAll [ ] = [ ]
doubleAll (head:tail) = 2*head : doubleAll tail
```

Section 4. Selecting Elements

```
doubleSome list = [2*x | x <- list, x < 10]
```

```
doubleSome (head:tail)
  | head < 10 = 2*head : doubleSome tail
  | otherwise = head   : doubleSome tail
```

Section 5. Combining Results

```
sumList :: [Int] -> Int
sumList [] = 0
sumList (head:tail) = head + sumList tail
sumSquares [] = 0
sumSquares (head:tail) = head*head + sumSquares tail
concatenate :: [[a]] -> [a]
concatenate [] = []
concatenate (head:tail) = head ++ concatenate tail
```

Exercises

Generate each list using extended list notation with [1..10] as generator:

16. [11,12,13,14,15,16,17,18,19,20]
17. [[2],[4],[6],[8],[10]]
18. [[10],[9],[8],[7],[6],[5],[4],[3],[2],[1]]
19. [True,False,True,False,True,False,True,False,True,False]
20. [(3,True),(6,True),(9,True),(12,False),(15,False),(18,False)]
21. [(5,False),(10,True),(15,False),(40,False)]
22. [(11,12),(13,14),(15,16),(17,18),(19,20)]
23. [[5,6,7],[5,6,7,8,9],[5,6,7,8,9,10,11],[5,6,7,8,9,10,11,12,13]]
24. [21,16,11,6,1]
25. [[4],[6,4],[8,6,4],[10,8,6,4],[12,10,8,6,4]]

Practice 3: Patterns, Tuples, Recursion and Extended List Notation

26. Implement divisors: takes an integer and returns a list of its divisors.
27. Implement prime: returns True if its single integer argument is prime (1 is not prime).
28. Create an expression using extended list notation that generates the primes between 1 and 100.
29. Find out how map and filter work, then implement them using extended list notation as myMap and myFilter.
30. Program evaluations :: [a] -> [(a->b)] -> [[b]]. Each sub-list contains results of applying all functions to one value. Main> evaluations [1,2,3] [double,triple] → [[2,3],[4,6],[6,9]]
31. Use the previous function to check [0,π/2,-π/2,π,-π]: sine>0, cosine=0, tangent=0.
32. Implement factorisation: returns prime factorisation as a list of (factor, exponent) tuples. Main> factorisation 60 → [(2,2),(3,1),(5,1)]

33. Implement `takeWhileMy` and `dropWhileMy` (analogous to `takeWhile/dropWhile`).
34. Implement `trimSpaces`: removes leading spaces from a string.
35. Review exercises and identify where currying and higher-order functions were used.

Practice 4: Haskell Exercises from Previous Exams

36. Implement a function that approximates the integral of a function over $[a,b]$ given precision t .
37. (February 2006) Implement a function that approximates π using a 2×2 square grid with spacing t . Count grid-cell centres inside the unit circle.
38. (November 2006) Implement a hash function: $f([a_1, \dots, a_n]) = a_1 \times p_n + a_2 \times p_{n-1} + \dots + p_1 \times a_n$, where p_i are the i -th prime numbers.
39. (September 2007) Implement set operations on LISTS WITHOUT REPETITION: `member`, `subset`, `equal`, `union`.

PRACTICAL SESSIONS — PROLOG

Practice 1: Introduction to the SWI-Prolog Environment

Section 1. Getting Started

- Select Windows XP at startup.
- From the Windows Start menu, select SWI-Prolog.

Section 5. Writing a Program

Use a plain-text editor (Notepad). Write and edit programs with:

```
1?- edit(file('new.pl')).
```

Use extension ".pl" for Prolog programs. Files are saved in "My Documents\Prolog". Predicates and constants begin with lower-case. Load with:

```
?- consult('new.pl').
?- listing.
```

After modifying a file, update with:

```
?- make.
```

Operators

"=" (unification): does not evaluate arithmetic. "==" (identity). "is" (arithmetic evaluator — evaluates right, unifies left). "=:=" (arithmetic comparator).

```
?- X = 1+1.    X=1+1    Yes
?- X is 1+1.   X=2
```

`?- 2 == 1+1. Yes`

Section 6. Variable Scope

The scope of a variable in Prolog is restricted to a single clause.

Section 7. Help Predicates

Query information about any predicate: `apropos(unification)`. Predicates are named as `predicate_name/arity`.

Section 8. Debugger

Enable graphical debugger: `?- guitracr`. Then trace with: `?- trace`. Step through with the space bar.

Exercises

1. Unification

Check whether the following terms unify:

- (a) $X=1$. (b) $X=\text{woman}(\text{maria})$. (c) $X=2+3$. (d) $X=1, X=2, X=Y$. (e) $X=2+X$.
(f) $X=1, Y=1, X=Y$. (g) $X=2, Y=1+1, X=Y$. (h) $X=3+4, Y=7, X=Y$. (i) $X=2*6, X=2*Y$.
(j) $5=2+3$. (k) $2+3=2+Y$. (l) $f(X,b)=f(a,Y)$. (m) $2*X=Y*(3+Y)$.
(n) $(1+2)=1+2$. (o) $1+1=+(1,1)$. (p) $(1+1)+1=1+(1+1)$.

2. Arithmetic and Unification

- (a) X is $2+3$, $X=5$. (b) X is $2+3$, $X=2+3$. (c) $2+3$ is X .
(d) X is 5 , 6 is $X+1$. (e) $X=5$, $6=X+1$. (f) 6 is $X+1$, $X=5$.
(g) $Y=2$, $2*X$ is $Y*(Y+3)$. (h) $Y=2$, Z is $Y*(Y+3)$.

Practice 2: Simple Predicates in Prolog

40. Implement `natural(X)`: true if X is a natural number.
41. Implement `factorial(Number, Result)`: true if `Result` is the factorial of `Number`.
42. Implement `fib(N, F)`: true when F is the N -th Fibonacci number. $\text{fib}(n)=\text{fib}(n-1)+\text{fib}(n-2)$.
43. Given dishes classified as starter/main/dessert with prices, write a predicate to find a complete menu costing less than N .
44. Implement `sum(X,Y,Z)` using Peano arithmetic: $0=0$, $1=s(0)$, $2=s(s(0))$, ...
45. Using the previous representation, implement `subtract(X,Y,Z)` and `product(X,Y,Z)`.
46. Make `fib(N,F)` reversible using Peano arithmetic: $\text{fib}(N,s(s(0))) \rightarrow N=s(s(s(0)))$.

Practice 3: List Predicates in Prolog

47. Predict and verify the results of: $[X|Y]=[a,b,c,d]$; $[X,Y|Z]=[a,b,c]$; $[X,Y|Z]=[a,b,c,d]$; $[X,Y,Z|A]=[a,b,c]$; $[X,Y,Z|A]=[a,b]$; $[X,Y,a]=[Z,b,Z]$; $[X,Y|Z]=[a,W]$.
48. Define recursive predicates for: (a) create list from 2 elements; (b) insert at beginning/end/position N; (c) concatenate two lists and implement prefix, suffix, sublist; (d) reverse; (e) delete one/all occurrences; (f) replace one/all occurrences; (g) palindrome of length $2n$; (h) shift right; (i) shift left.
49. Define `divide(+N,+OrigList,-Greater,-Smaller)`: Greater contains elements $> N$, Smaller contains elements $\leq N$.
50. Define `merge_sorted(+L1,+L2,-Result)`: merge two sorted lists into a sorted result.
51. Define `sort(List, R)`: true if R contains the sorted elements of List. Implement via (a) permutations; (b) split, sort, and merge.
52. Write `prod(+L, ?P)`: P is the product of elements of integer list L.
53. Write `dotProduct(+L1,+L2,-P)`: P is the dot product of vectors L1 and L2. Fail if different lengths.
54. `divisors(+N, ?L)`: L is the sorted list of divisors of N. `divisors(6,[2,3,6,1])` must answer NO.
55. `permute(+L,-P)`: P is a permutation of L. Allow repeated permutations if L has repeated elements.
56. Set operations on lists without repetition: (a) member; (b) insert; (c) union; (d) intersection; (e) difference; (f) convert list-with-repetitions to set.
57. Multi-set operations on (element,multiplicity) pairs: (a) member; (b) compute multiplicity.

Practice 4: Trees in Prolog

Binary Trees

58. Implement `count_nodes/2`: count nodes in a binary tree.
59. Implement `count_internal/2`: count internal nodes (with at least one child).
60. Implement `count_leaves/2`: count leaf nodes (with no children).
61. Implement `sum_nodes/2`: sum the content of all nodes.
62. Write `member/2`: true if element X belongs to the tree.
63. Implement `equal/2`, `symmetric/2`, and `isomorphic/2` for two trees.
64. Implement `depth/2`: true when the second argument is the depth of the tree.
65. Implement `balanced/1`: true when the tree is balanced.
66. Implement traversal predicates: `in_order/2`, `pre_order/2`, `post_order/2`, `breadth_first/2`.

Generic Trees

Repeat the previous exercises (except `in_order`, `pre_order`, `post_order`) for generic trees instead of binary trees.

Binary Search Trees

- `create_empty_tree(X)`: returns an empty tree.
- `insert(E,T,NewT)`: inserts E into T, returning NewT. If E already exists, return T unchanged.
- `height(T,X)`: returns the height of T.
- `balanced(T)`: true if T is balanced.
- `traversals(T,Pre,Post,In)`: returns pre-order, post-order, and in-order traversals.

Implement a predicate that builds a BST from a list of integers:

```
?- build([3,2,5,7,1], T).  
T = t(3, t(2, t(1, nil, nil), nil), t(5, nil, t(7, nil, nil)))
```

Practice 5: Graphs in Prolog

67. Write a Prolog program that traces an envelope figure without lifting the pen — traverse each edge exactly once.
68. Represent a road-map graph and find all paths between Madrid and Oviedo including road names and distances. Extend with two extra edges and handle cycles by using a visited list.
69. Create a Prolog program with predicates:
 - `path(G,N1,N2,Path)`: find all paths between N1 and N2 via backtracking.
 - `distance(G,N1,N2,D)`: compute distance; two versions (with/without cycles).
 - `cycles(G,Cycle)`: find cycles — a path from a node back to itself.
 - `connected(G)`: true if all nodes are reachable from all other nodes.
 - `degree(G,N,Deg)`: the number of nodes N is connected to.
 - `list_degrees(G,L)`: all nodes with their degrees.
 - `colour_graph(G,ColourList,NodeColours)`: colour graph so no adjacent nodes share a colour.